

ALTIS

Operator's Manual

AI-powered flood damage triage for property & casualty insurers. This guide covers everything from first launch to running a full investor demo, plus how the science works and what the system can and cannot do today.

Version	MVP build — branch <code>claude/wizardly-volta-bj8wdd</code>
Audience	Operators, demo presenters, evaluators
Two processes	FastAPI backend (port 8000) + React frontend (port 5173)
Keys needed	Mapbox token (globe + global geocoding). Earth Engine key = live global analysis.

1 — What Altis is

After a major flood, a carrier faces thousands of claims and a fixed number of adjusters. Sending someone to every property wastes days on homes that were barely touched, while the worst-hit wait. Altis turns satellite radar into a property-level triage within hours of the event — before anyone drives anywhere — telling you who to dispatch today, who can be resolved remotely, and how confident it is in each call.

What it produces, per property

- A **triage decision**: Dispatch, Remote-Approve, Remote-Deny, or Review.
- An **estimated flood depth** with an uncertainty interval ($\pm$ feet).
- A **calibrated confidence** score, validated against FEMA declarations.
- A plain-English “**why this decision**” breakdown of every factor.

The five things a demo should show

- The **3D globe** flying to an event with ~1,000 triaged properties.
- A **severity-ranked dispatch queue** — the worklist, not a flat list.
- **Universal portfolio upload** — any messy carrier spreadsheet, fuzzy-mapped.
- The **adjuster feedback loop** — a human correction that re-trains the model.
- A one-click **audit-ready PDF** a carrier could file.

2 — How the system fits together

Three independent units. For a local demo you only run the first two; the monitor is optional and needs internet.

Unit	What it is	Where	Needed for demo?
Frontend	React + Mapbox GL globe UI	frontend/	Yes (port 5173)
Backend	FastAPI + SQLite API	backend/	Yes (port 8000)
Pipeline	Sentinel-1 SAR flood detection	pipeline/	Pre-computed; no
Validation	FEMA calibration + precision/recall	validation/	Optional
Monitor	NHC/USGS event detection	monitor/	Optional

The frontend talks to the backend only through `/api/` . . . The Vite dev server proxies that to port 8000 automatically, so you never configure URLs for local use.

3 — First-time setup

Prerequisites

- **Python 3.10+** and **Node 18+** (check: `python --version`, `node --version`).
- A free **Mapbox token** from account.mapbox.com (free tier = 50,000 map loads/month).
- Git, and a clone of the repo on the `claude/wizardly-volta-bj8wdd` branch.

Step A — Backend (run from the repo root)

```
python -m venv .env
.venv\Scripts\Activate.ps1 # Windows PowerShell
# (macOS/Linux: source .env/bin/activate)
pip install -r requirements.txt
```

PowerShell note: if activation is blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` once, then re-run the activate line. When active, your prompt starts with `(.env)`.

Step B — Start the backend

```
uvicorn backend.main:app --reload --port 8000
```

You should see `Altis backend ready` at `http://localhost:8000`. Leave this window running.

Step C — Frontend (new terminal. from frontend/)

```
cd frontend
npm install
copy .env.example .env # macOS/Linux: cp .env.example .env
# open .env and paste your token:
# VITE_MAPBOX_TOKEN=pk.your_real_token
npm run dev
```

Open **`http://localhost:5173`**. The globe should render and slowly rotate. If it's black, the token isn't loaded — see [Troubleshooting](#).

4 — The interface, panel by panel

Left sidebar

Events	Pick the flood event. The globe flies there and loads properties.
Analysis	Score an uploaded portfolio against the selected event.
Dispatch Queue	Severity × coverage-ranked worklist — who to send first.
Operations	Monitor → pipeline run queue. Queue and advance runs.
Reports	Calibration reliability chart + the audit PDF download.

Top bar

Upload Portfolio	Bring in a carrier spreadsheet (.csv/.xlsx/.xls/.pdf).
Claims Grid	Full-screen sortable/filterable table with bulk CSV export.
KPI strip	Properties, dispatch count, remote-resolved, estimated savings.

The globe

- Pins are color-coded by decision: **red = Dispatch**, **green = Remote-resolved**, **amber = Review**.
- Red Dispatch pins **glow and grow** as you zoom in; they're emphasized on purpose.
- **Addresses label** only when you zoom in close, to keep the wide view clean.
- Click any pin to open the property drawer on the right.

5 — Core workflows

5.1 Inspect a single property

- 1 Click any pin on the globe.
- 2 The right drawer shows the SAR before/after, estimated depth ($\pm$ uncertainty), the confidence gauge, and the “Why this decision” factor breakdown.
- 3 Scroll to **Adjuster Verdict**: click ☐ Agree or ☐ Disagree, optionally pick a corrected class and add a note, then Submit. This writes to the ground-truth table.

5.2 Work the dispatch queue

- 1 Sidebar → **Dispatch Queue**. Rows are ranked by severity $\times$ coverage exposure.
- 2 #1 is what the CAT team hits first. Click a row to inspect that property.
- 3 Use **Open grid** → to jump into the full table view.

5.3 Claims data grid

- 1 Top bar → **Claims Grid** (or “Open grid” from the queue).
- 2 Sort by any column, filter by triage decision, or search.
- 3 Tick rows and **Export selected** (or Export filtered) to CSV — the manager's spreadsheet-native view.

5.4 Upload a portfolio (the headline feature)

- 1 Top bar → **Upload Portfolio**. Drag in a .csv/.xlsx/.xls/.pdf — any column names.
- 2 Altis fuzzy-maps your columns to Policy / Address / Coverage / City / State / Zip and shows a **review-and-confirm** screen, color-coded by match confidence (teal = high, amber = medium, dim = unmatched).
- 3 Override any wrong mapping from the dropdowns; check the preview rows and the flagged-address callout.
- 4 Click **Confirm & Geocode**. The globe flies to your geocoded book.
- 5 Sidebar → **Analysis** → analyze against the matching event, then revisit Dispatch Queue in *portfolio* mode.

Try the included samples in `samples/`: `demo_portfolio_harvey.csv` and `demo_portfolio_charlotte.csv` have deliberately messy, real-world carrier headers. **Match the sample to its event** — Harvey samples → Hurricane Harvey, Charlotte/Ian samples → Hurricane Ian.

5.5 Audit PDF + validation

- 1 Sidebar → **Reports** → **Download audit PDF**: a carrier-ready document with methodology, satellite scene sources + dates, the triage table, top dispatch priorities, and FEMA precision/recall.
- 2 The reliability chart appears once you've generated calibration (see §7).

5.6 Operations — the always-on loop

- 1 Sidebar → **Operations** shows the monitor → pipeline run queue.
- 2 Click **+ Queue run** to enqueue one manually; click a status pill to advance it (queued → running → complete).

6 — How the triage actually works

This is the section that wins technical due-diligence. Altis is not a black box — every decision is traceable to physical signals.

Detection

- **SAR change detection.** Sentinel-1 radar sees through clouds (which blanket every hurricane). Altis compares *before* vs *after* backscatter, with a speckle filter and an Otsu threshold guarded to a physical open-water range.
- **Optical cross-check.** When a cloud-free Sentinel-2 view exists, MNDWI confirms or contradicts the radar call — advisory only, never blocking.
- **Depth.** Water-surface elevation minus ground elevation from a DEM, clamped to physically plausible residential depths.

Trust & safety

- **Uncertainty interval.** Depth is reported $\pm$ a 1σ interval driven by the DEM's own vertical accuracy — never shown more precise than the elevation data supports.
- **Ensemble disagreement → Review.** Three members vote (SAR, optical, DEM-hydrology). When they genuinely conflict, the property is sent to manual Review rather than a confident automated call.
- **Calibration.** Raw scores are isotonic/Platt-calibrated so a “78% confidence” means what it says, validated against real FEMA disaster declarations.
- **Human-in-the-loop.** Adjuster verdicts (§5.1) become per-property ground truth that overrides coarse zip-level FEMA labels and feeds the next calibration run.

7 — Optional: real validation & live detection

Both need internet but no API key.

FEMA validation + calibration

```
python validation/accuracy_check.py --event harvey
```

Writes outputs/validation_harvey.md and outputs/calibration_harvey.json, which populate the Reports reliability chart and the PDF's precision/recall section. Any adjuster verdicts you submitted are merged in as property-level ground truth.

Live event detection

```
python monitor/monitor.py # one pass; --loop to run continuously
```

Detected NHC/USGS flood events are auto-queued as pipeline runs and appear in the Operations panel — the monitor → pipeline loop, closed.

Live GLOBAL satellite analysis (any location on Earth)

With a Google Earth Engine service-account key in place, Altis runs real Sentinel-1 flood detection on demand for any uploaded portfolio, anywhere — no pre-computed event needed. Drop the key at `secrets/ee-sa-key.json` (or set `GEE_SERVICE_ACCOUNT_KEY`) and restart the backend.

- 1 Upload a portfolio (Upload Portfolio → map columns → confirm). Files with **latitude/longitude columns geocode instantly**; address-only files use the global Mapbox geocoder.
- 2 Open the **Analysis** panel, switch source to your portfolio. A “Live satellite analysis” card shows a **Global** badge when Earth Engine is connected.
- 3 Enter the **flood/landfall date** and click **Run**. Altis composites Sentinel-1 before/after that date, detects flooding, and triages every property (30–90s).
- 4 Open any property — the SAR/Optical panel now shows **real Sentinel imagery** for that exact spot and date, with the unmistakable bright-to-black flood signature.

Try it: upload `samples/demo_portfolio_pakistan_floods.csv` and run the date **2022-09-05** (the catastrophic Sindh floods). Most properties come back Dispatch with multi-foot depths — real Sentinel-1 detection over Pakistan.

A second, independently verified dataset is included:

`samples/demo_portfolio_australia_floods.csv` — run the date **2022-02-28** (the record Northern Rivers/Lismore floods, NSW). 14 of 16 properties come back Dispatch with depths up to 5.6ft; the Lismore CBD parcel correctly routes to Review (dense-urban ambiguity) and the rural high-ground control to Remote-Deny. Both datasets were built by grid-sampling the real flood extent live via Earth Engine first — never by guessing coordinates.

Where SAR shines vs. struggles: it is strongest on **open-water, riverine, and storm-surge flooding** (vast, unambiguous dark water). It is weakest on **dense-urban rainfall flooding**, where buildings hide street-level water and floods often recede before the satellite overpass. Altis is honest about this — borderline cases go to Review with the confidence score attached, not a false Dispatch/Deny.

8 — Coverage, limits & honest answers

Read this before a demo. These are the questions a sharp evaluator asks, and the true answers.

“Does it work anywhere?”

- **Yes, globally — and it's wired live.** Sentinel-1 images the entire Earth every ~6–12 days. With an Earth Engine key, uploading a portfolio and entering a flood date runs real detection for that location on demand (see §7) — proven on the Pakistan 2022 floods, not just US events.
- **Geocoding is now global** via Mapbox (Colombia, Pakistan, anywhere); the US Census geocoder remains a fallback. Files with lat/lon columns skip geocoding entirely.
- **Global elevation:** the pipeline uses US 3DEP lidar where available, then Copernicus GLO-30 (worldwide 30m), then SRTM — so depth works outside the US too.
- The two pre-computed events (Harvey, Ian) still ship for instant, no-login demos.

Real constraints today (don't oversell)

- **You need an actual flood with known dates.** SAR compares before vs after, so you can't point at a dry location on a random day and get a result.
- **SAR has a sweet spot.** Open-water / riverine / storm-surge flooding detects beautifully; dense-urban rainfall flooding is genuinely hard (buildings hide water, floods recede before the overpass). Altis routes those to Review rather than guessing.
- **Depth precision tracks the DEM.** Outside the US (GLO-30 30m, ~6m vertical error) depth intervals are wide — surfaced honestly as $\pm$ feet, never hidden.
- **Calibrated confidence is sharpest where there's ground truth.** FEMA anchors US calibration; new markets sharpen as partners feed claims back through the adjuster loop.
- **Without an Earth Engine key,** live analysis is disabled (the app says so via `/api/gee-status`) and imagery falls back to synthetic previews — the demo never hard-fails.

The defensible one-liner: *“Real global flood detection on day one — strongest on open-water flooding, honest about urban and depth limits, and it gets sharper per market as partners feed ground truth back.”*

9 — Adding a new flood event

- 1 In `pipeline/config.py`, copy the HARVEY dict to a new one (e.g. MILTON) and set: `event_id`, `label`, the `[west, south, east, north]` bbox, and the `pre_/post_` date windows around the flood.
- 2 Register it in the `EVENTS` dict at the bottom of the file.
- 3 Set up Google Earth Engine (service account) and run `python pipeline/03_flood_pipeline.py` for the new event to generate its outputs/ CSVs.
- 4 Restart the backend. The new event appears in the sidebar automatically.

For production hosting (Vercel + Render, Postgres migration, GEE service account, hardening checklist, ~\$10–30/mo pilot cost), see `DEPLOYMENT.md` in the repo.

10 — Troubleshooting

Symptom	Fix
Globe is black / “Set VITE_MAPBOX_TOKEN” in console	Put a real token in frontend/.env, then restart npm run dev.
Frontend loads but no data appears	Make sure the backend is running on port 8000 (the dev server proxies /api to it).
Upload says “could not geocode”	The Census geocoder needs internet and real US addresses.
PDF / report 404	Generate calibration first (§7) for the validation section; the rest works regardless.
PowerShell blocks .venv activation	Run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then activate again.
Port already in use	Stop the other process, or change --port (and the proxy target in vite.config.js).

Quick reference also lives in the repo: RUN_LOCAL.md (setup + demo script), DEPLOYMENT.md (hosting), and README.md (overview + API surface).